Mosaic: GPU Sharing with Latency Guarantees through Kernel-Level Interference Prediction

ETH Zurich Paper
from team who did Understanding GPU Resource Interference One Level Deeper

Recap on GPU Interference: CUDA Programming

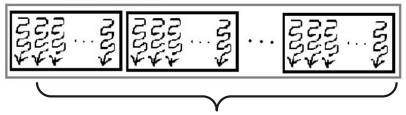
CUDA Kernel

```
__global__ void vecAdd(const float* A, const float* B, float* C, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) C[i] = A[i] + B[i];
}
int main() {
    // Launch: <<< gridSize, blockSize >>>  ->  4096 blocks x 256 threads
    vecAdd<<<4096, 256>>>(d_A, d_B, d_C, n);
}
```



Send to GPU

Kernel Grid

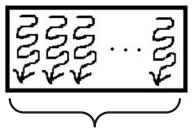


4096 Blocks
(defined by kernel)



Decomposed to Blocks, scheduled by the GPU Block Scheduler

Thread Block

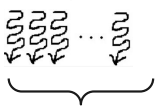


256 threads
(defined by kernel)



Assigned to one SM until finished

Warp



32 threads
(fixed)



Assigned to one SMSP until finished

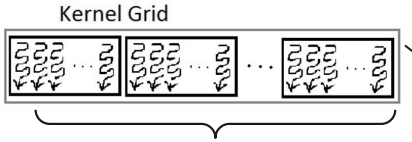
Thread



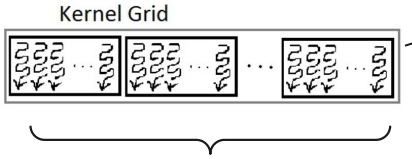


Executed on the execution lanes in

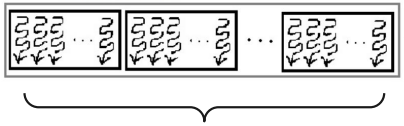
Single Kernel On GPU



90 Blocks



180 Blocks



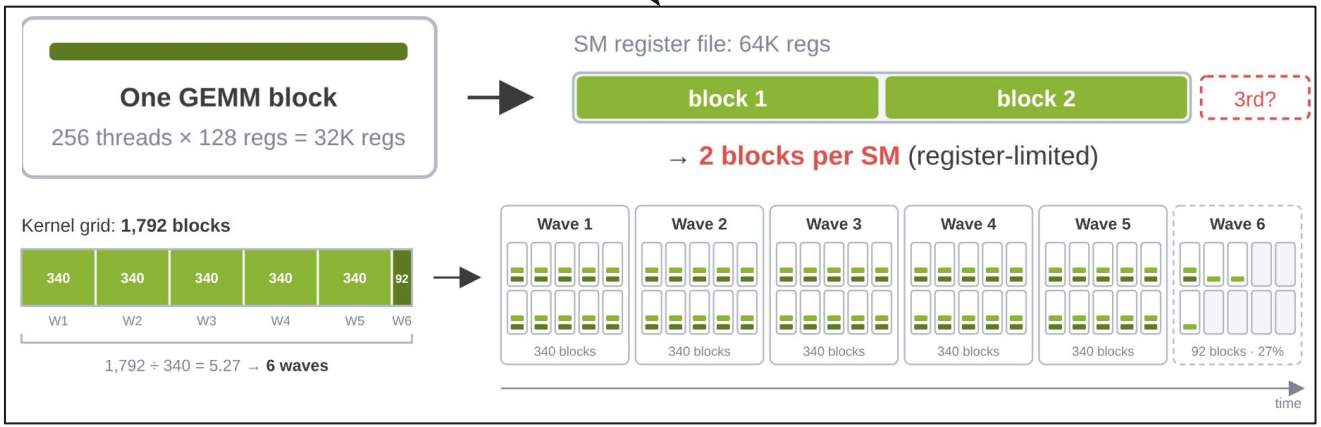
1,792 Blocks





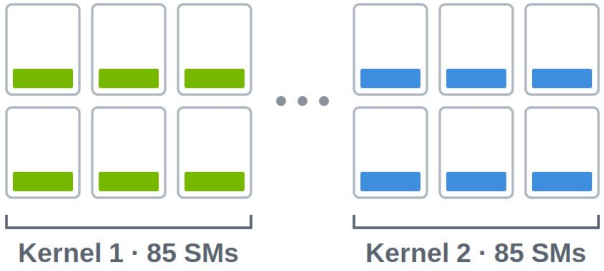


Multiple Blocks can operate on one SM



Kernel Colocation On GPU

Kernels with SM Partitioning

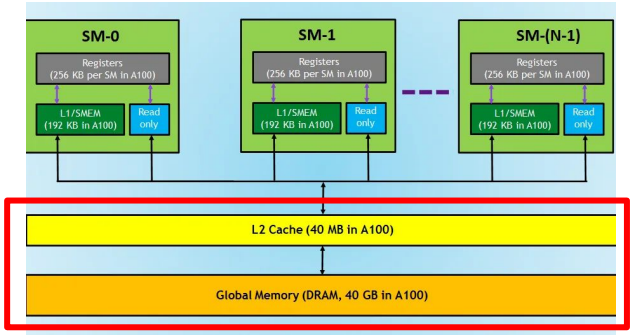


Case 1: Each kernel has only 85 blocks



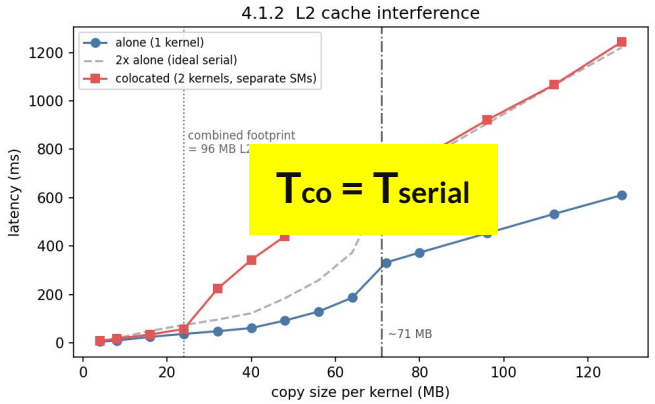
Case 2: partition GPU using MPS or Green Context

→ Inter-SM interference



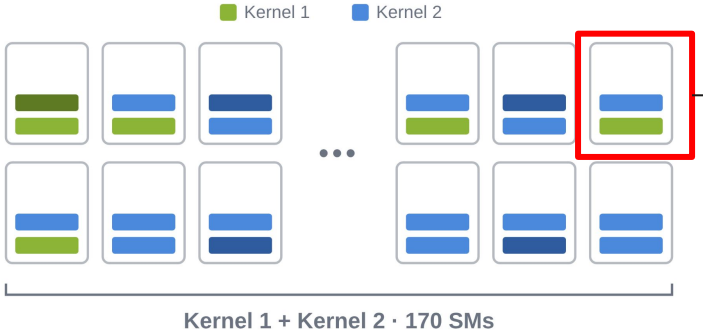
SMs share:

- DRAM bandwidth
- L2 Cache



Kernel Colocation On GPU

Kernels without SM Partitioning



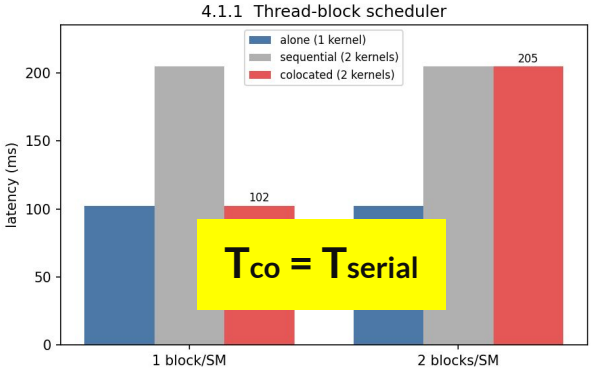
Shares

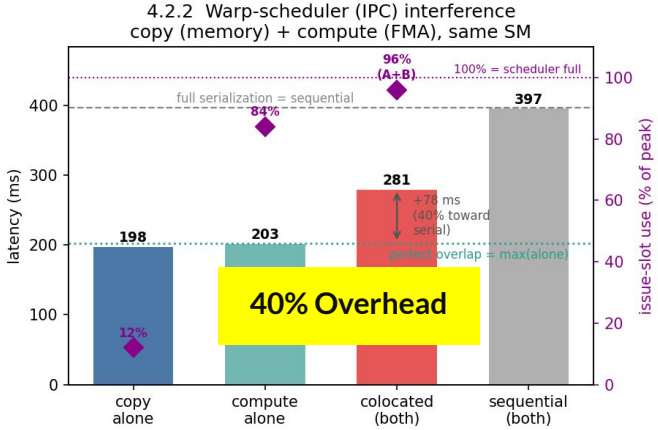


→ Inter + Intra-SM interference

Blocks in the same SM shares

- exec pipeline
- warp scheduler
- L1 Cache
- accelerators



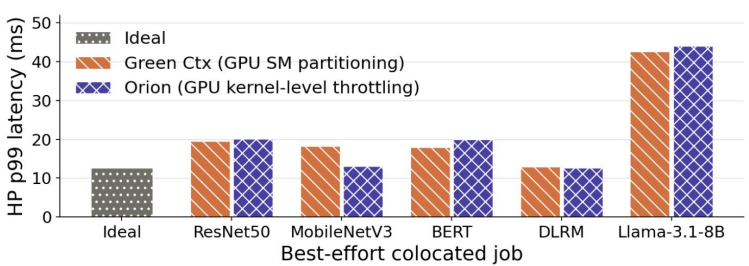


Why Existing Colocation Schedulers Fall Short

Heuristic-based schedulers: colocate kernels with complementary resources

Examples: Orion, Usher

- Use coarse resource metrics to select kernels
- Miss interactions across scheduling, and hardware
- Cannot estimate the latency impact of admission



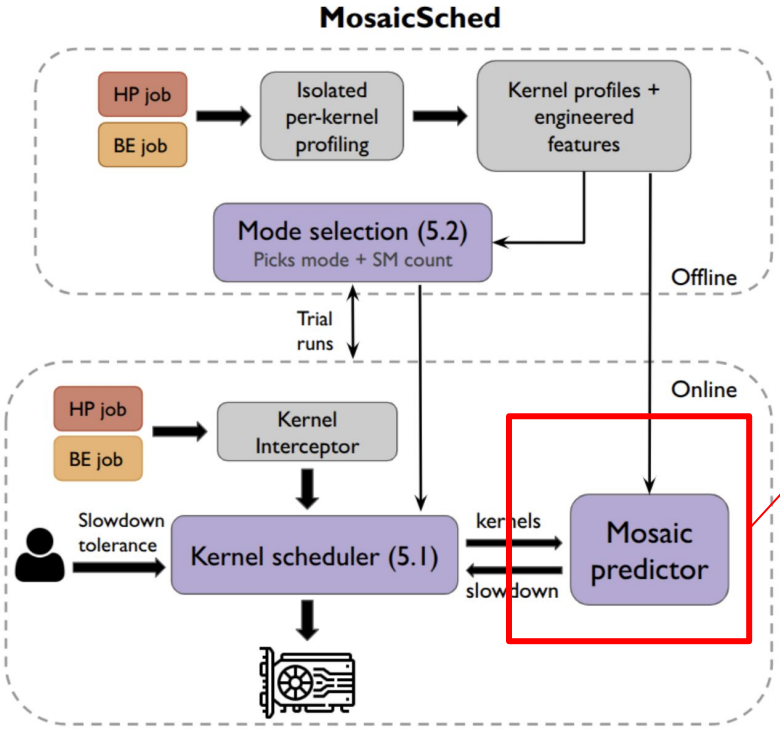
Existing interference predictors: estimate slowdown from kernel profiles to guide colocation

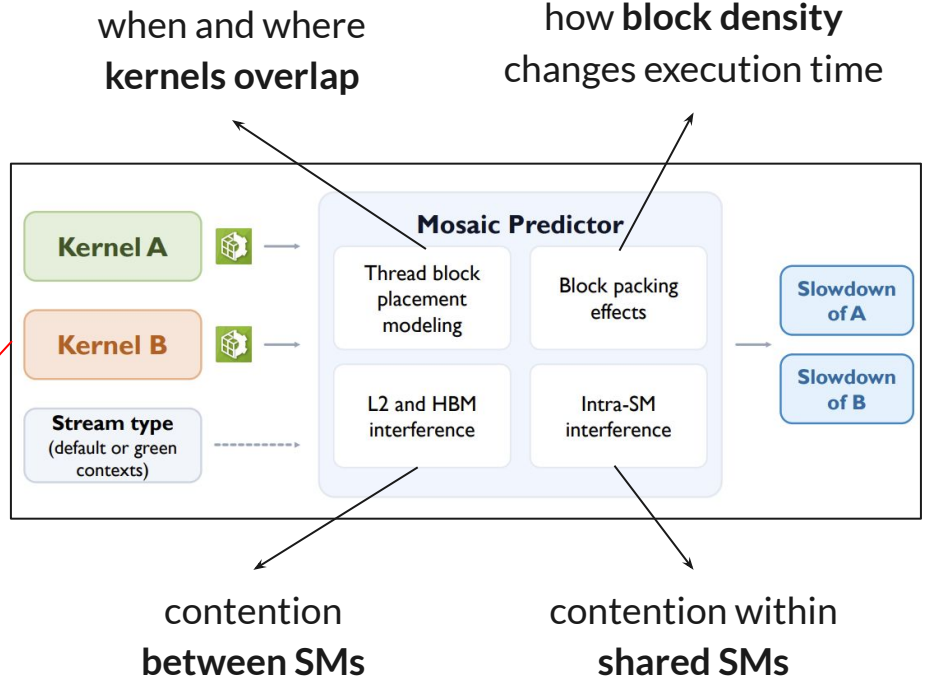
Examples: KACE, GPUPool

- Rely on coarse hardware metrics or simulator-specific features
- Do not explicitly capture block placement between kernels
- Miss memory-access behavior and latency-hiding capability

Mosaic: proposing a **kernel-level predictor** that captures both **execution overlap** and **resource contention** using profiles from real GPUs

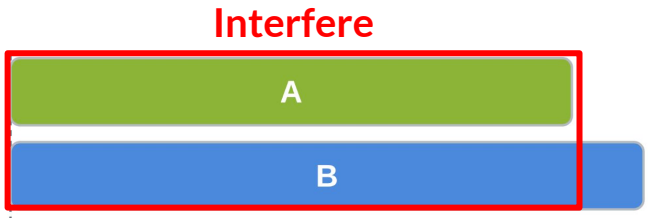
Mosaic Overview



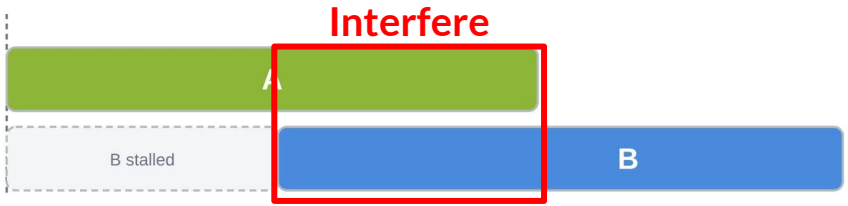


Step 1: Thread Block Placement

Kernels are launched concurrently on two equal-priority streams. What happens on the GPU?

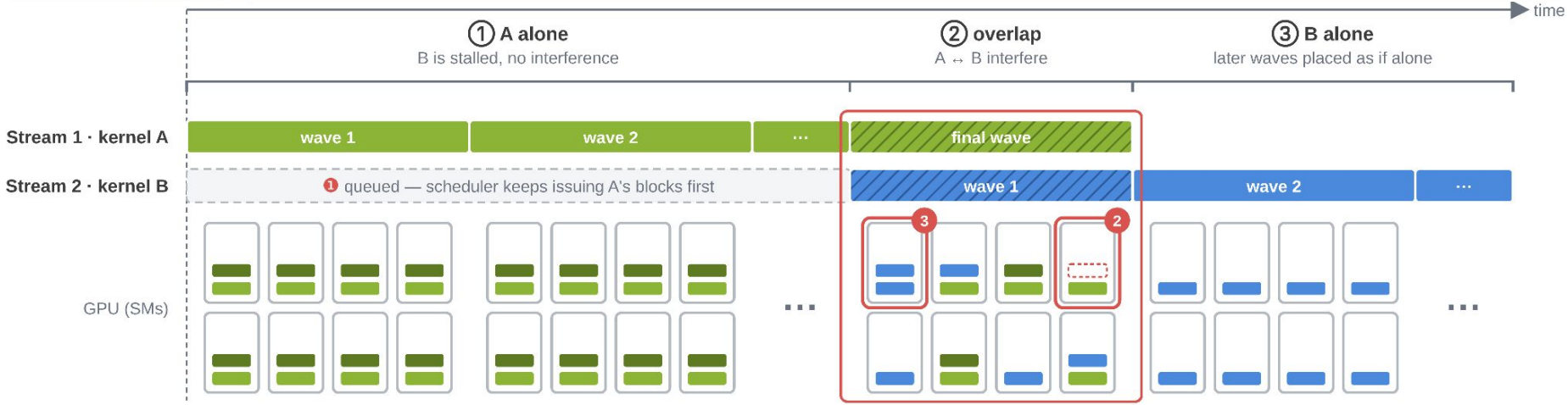


What you might think…



What is observed

What actually happened:



Thread Block Observations

Observation ❶: B can overlap only with A's final wave

GPU hardware scheduler tends to dispatch all of A's blocks before dispatching any of B's blocks.



Observation ❷: A free slot ≠ B can enter, depending on SM resources.

Observation ❸: overlap reshapes B's placement, B's per-SM block density ends up **different from running alone**

Mosaic's Placement Model

- Models block placement under SM resource constraints
- Uses offline SM timing statistics to estimate gradual resource release
- Predicts overlap timing, block distribution, and warp shares

Step 2: Modeling block packing

How does Observation ❸ alone impact kernel B's execution time?

1. **Higher per-SM block density**: Increases resource contention within an SM
2. **Fewer active B blocks across the GPU**: reduce B's own memory-system contention

Mosaic model this as

$$t_w^c = t_w^{iso} \underbrace{\left(\frac{k^c}{k^{iso}}\right)^{\beta}}_{\text{① Per-SM block density}} \underbrace{\left(\frac{X^c}{X^{iso}}\right)^{\gamma}}_{\text{② Total active B blocks}}$$

t_w: **wave execution time** k: **resident B blocks per SM** X: **total active B blocks**

iso / c: **isolated / colocated execution** β, γ: **fitted sensitivity parameters**

Main idea: scales isolated wave time to account for both placement effects.

Step 3: Modeling Inter-SM Interference

Recap: on separate SMs, kernels contend for shared L2 and DRAM resources

Observation 1: Average bandwidth utilization alone cannot explain interference

a. Read and write cause different slowdowns
b. Averages hide temporal variation

Observation 2: Different kernel tolerate interfere differently

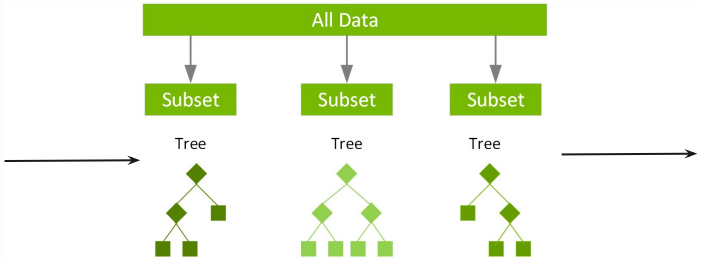
a. Similar memory profiles can lead to different slowdowns
b. Kernels has different capability in overlap memory access with compute

Mosaic predict the slowdown of two kernels using a lightweight **XGBoost** decision tree model

Input:
Both kernels'

- SM allocations,
- memory traffic
- latency-hiding metrics

from isolated profiling



XGBoost

Output:
Predicted inter-SM slowdown factors S_A^{inter} S_B^{inter}

where

$$t_A^{\text{co}} = \mathbf{S}_A^{\text{inter}} \cdot t_A^{\text{iso}}$$

$$t_B^{\text{co}} = \mathbf{S}_B^{\text{inter}} \cdot t_B^{\text{iso}}$$

Step 4: Modeling Intra-SM Interference

Mosaic groups intra-SM contention into three modeling components:

Factor	Mosaic's Modeling	Reason
Execution pipelines	**Analytical model** S_{pipe} using pipeline demand and warp shares	Overlapping pipeline demands create resource contention
Warp scheduling	**Analytical model** S_{ipc} using IPC differences and memory stalls	Issue intensity and memory readiness affect scheduling contention
Memory subsystem	**ML** Δ_{ML}: XGBoost residual model	Cache, shared-memory, and LSU/MIO interactions are difficult to model analytically

Therefore predicting B's slowdown due to A is:

$$S_{B\leftarrow A}^{\text{analytical}} = \max\left(S_{B\leftarrow A}^{\text{pipe}}, S_{B\leftarrow A}^{\text{ipc}}\right)$$

$$\mathbf{S}_{B\leftarrow A}^{\text{intra}} = S_{B\leftarrow A}^{\text{analytical}} + \Delta_{B\leftarrow A}^{\text{ML}}$$

where

$$S_{B\leftarrow A}^{\text{pipe}} = \begin{cases} 1, & \sigma_B > \sigma_{\text{th}}, \\ \max\limits_{P} \dfrac{r_B^P}{\min\left(r_B^P, \max\left(w_B, 1 - r_B^P\right)\right)}, & \text{otherwise.} \end{cases}$$

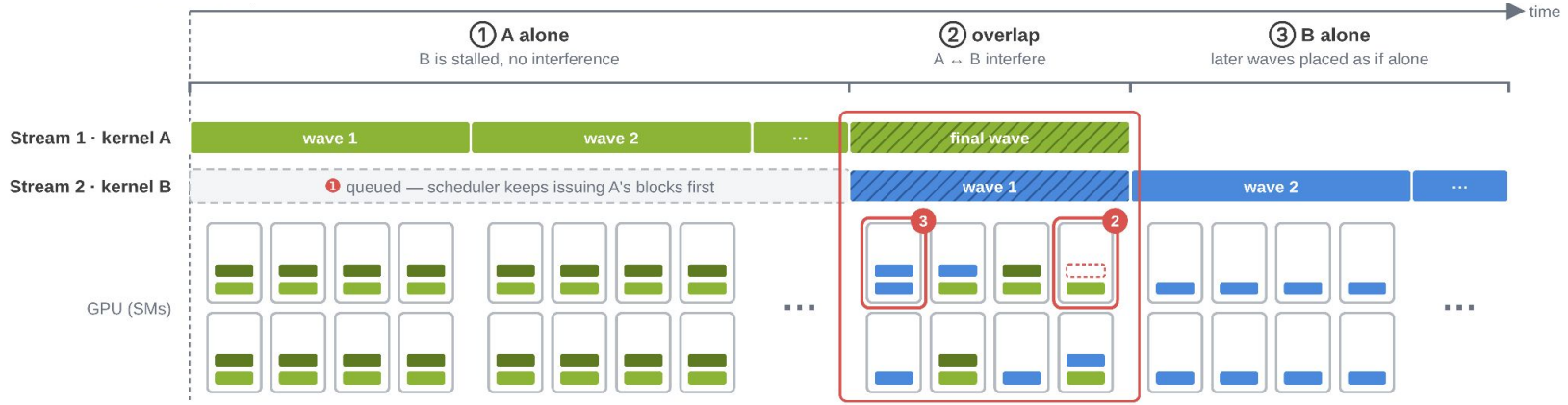
$$S_{B\leftarrow A}^{\text{ipc}} = 1 + \frac{k}{1 + k_\sigma \sigma_B} \max(0, IPC_A - IPC_B)$$

Pipeline model: Estimates slowdown from pipeline demand and available capacity

IPC model: Estimates scheduling slowdown from IPC differences, adjusted for memory stalls

Data Source: Hardware metrics from isolated offline profiling, placement modeling, and GPU-specific calibrated coefficients

Step 5: Putting Everything Together



1. **Determine overlap:** placement model predicts when and where kernels execute concurrently
2. **Estimate overlap slowdown:** combine inter-SM and intra-SM predictions
3. **Reconstruct execution time:** account for both overlapping and non-overlapping phases

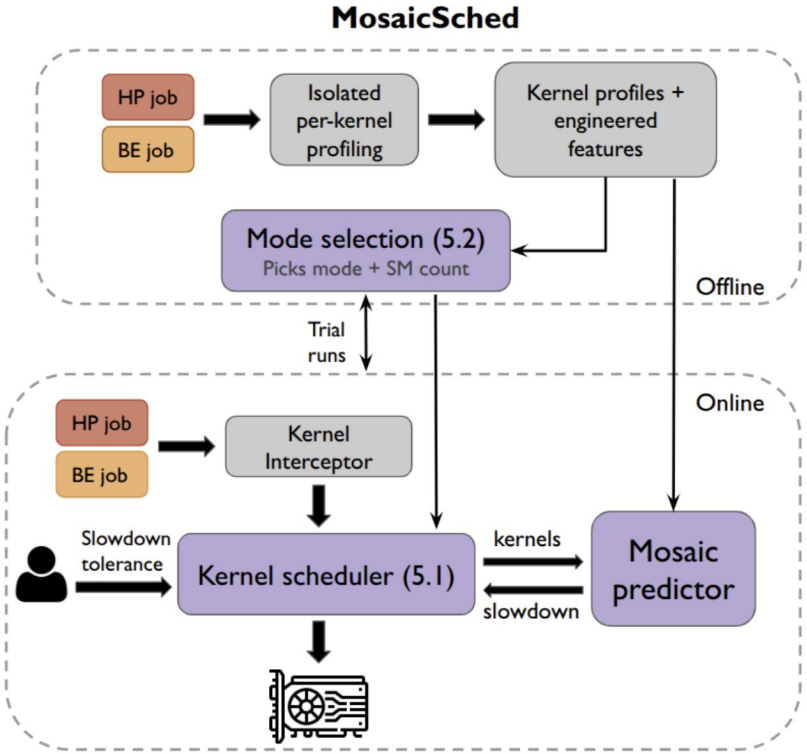
$$S_{B \leftarrow A} = \max\left(S^{\text{inter}}_{B \leftarrow A}, S^{\text{intra}}_{B \leftarrow A}\right)$$

$$t^{\text{overlap}}_{w,B} = S_{B \leftarrow A} \cdot S^{\text{pack}}_{B} \cdot t^{\text{iso}}_{w,B}$$

SM partitioning case: only inter-SM contention applies, using isolated execution times with the same SM allocations

MosaicSched: Prediction-Based Kernel Admission

Target: keep HP latency within user-defined target, maximizing BE throughput



1. Dispatch HP kernels immediately
2. Predict the impact of each candidate BE kernel on HP completion time
3. Admit the BE kernel only if the predicted completion time stays within budget:

$$\underbrace{T_{\text{completed}} + T_{\text{inflight}} + T_{\text{overlap}}(b) + T_{\text{remaining}}}_{\text{Predicted HP completion time}} \leq \underbrace{p' \cdot T_{\text{iso}}}_{\text{HP latency budget}}$$

Offline profiling selects **full-GPU sharing** or an **SM partition** to maximize BE throughput under the HP latency target

Brief Evaluation: Predictor Accuracy on H100

Execution Mode	Predictor	p50 Error ↓	p95 Error ↓
Separate SMs	KACE	19%	51%
	GPUPool	11%	45%
	Mosaic	**5%**	**23%**
Shared SMs	KACE	286%	1181%
	GPUPool	169%	1482%
	Mosaic	**8%**	**48.2%**